

School of Computer Science, University of Windsor
COMP-2540: Data Structures and Algorithms
Term: Summer 2026
Instructor: Dr. Asish Mukhopadhyay

Lab 9

Posted: 12 July, 2026

Due: 18 July, 2026

Instructions:

- You are expected to finish the lab by the end of the posted date. Submissions beyond the due date will earn a penalty of $n * 25\%$, where $n = submissionDay - dueDay$. Thus if the lab is due Tuesday and you submit on Wednesday, this will be considered a day late.
- Whether or not you finish your work during the lab hour you will have to upload your work on BLACKBOARD for record-keeping and grading before the beginning of the next lab. Create a script file as follows:
 1. `script LabName.txt`
 2. `cat LabName.c`
 3. `cat input.txt`
 4. `cc labName.c`
 5. `./a.out < input.txt`
 6. `ls -l`
 7. `exit` (DO NOT FORGET THIS STEP!!)
- There will be no make-up for missed labs. If you have missed a lab for truly extenuating circumstances (like illness or family emergency) I will consider allowing you to make a late submission. However, I need to be informed by email about this on the day of the missed lab. The email should include your name and SID.

Problem:

Implement in C the heapSort algorithm discussed in class.

For the BUILDHEAP procedure use the bottom-up approach.

Validate your heapsort program on arrays of small size (10's of elements) and compare the running times of your heapsort program with your implementation of the quicksort program (lab 7) on large integer arrays (at least 1000 entries).

To aid you further in completing the assignment, here is a framework that you might want to follow to write a complete and correct program:

```
//includes

//function prototypes
void buildHeap(int a[], int);
void heapSort(int a[], int);
void heapify (int a[], int, int);

int main(void){
int listSize,....

// read in the size of list to be created and allocate store
int *list = (int *)calloc(listSize, sizeof(int));

// populate array with random integer values

} // end of main

// build a heap
// following the pseudocode given in the slides

void buildHeap(int a[], int n){

} //end buildHeap

// heapsort

void heapSort(int a[], int n){

} // end heapSort

// heapify the elements in the index range i to j
// following the pseudocode given in the slides

void heapify (int a[], int i, int j){
// important note: children of a node with index i have indices 2i+1 and 2i+2
// since array indices start at 0
```

```
} //end heapify
```

(10 points)